دفتر 12 / من الشرح للتطبيق

نبني قائمة جديدة بوضوح

عايز تضاعف كل رقم أو تختار القيم المناسبة؟ ابدأ بفهم حلقة عادية، وبعدها اختصرها لصيغة مقروءة. الهدف إن القارئ يلاحظ التحويل والشرط بسرعة.

في آخر الدفتر هتقدر تعمل إيه؟

تكتب قائمة ناتجة من تحويل أو فلترة، وتفرق بين شرط اختيار العنصر والتعبير الشرطي الذي يحوله.

هنمشي بالترتيب ده

1. نحول for وappend إلى comprehension.
2. نضيف شرطا لاختيار العناصر.
3. نراجع الوضوح وحجم القائمة الناتجة.

قبل ما تبدأ

الحلقات والشروط وappend. لو الأمثلة المختصرة مش واضحة، ارجع للصيغة الطويلة.

افتح ملف كود جنب الدفتر. اقرأ المثال، توقع النتيجة، وبعدها شغله. لو النتيجة مختلفة، ارجع للسطر اللي غيرها بدل ما تعيد نسخ الكود كله.

خريطة الدفتر: الفكرة ص 2 • تطبيق خطوة بخطوة ص 3 • الأخطاء ص 4 • تمارين ص 5 • حلول ص 6 • مراجعة ص 7

بايثون مع نوار

List comprehensions

شرح • أمثلة • تمارين • حلول

12

01 / من الطويل للمختصر

نفس الفكرة في شكلين

PYTHON

```python
numbers = [1, 2, 3]
squares = []
for number in numbers:
    squares.append(number ** 2)
print(squares)
```

OUTPUT

```
[1, 4, 9]
```

بنبدأ بقائمة فارغة، ونحسب مربع كل عدد، ثم نضيفه. في comprehension بنضع التعبير الذي نريد إضافته أولا، وبعده for التي تحدد مصدر العناصر.

PYTHON

```python
numbers = [1, 2, 3]
squares = [number ** 2 for number in numbers]
print(squares)
```

OUTPUT

```
[1, 4, 9]
```

اقرأها كده: «مربع كل number في numbers». الناتج قائمة جديدة، والقائمة الأصلية لم تتغير. اسم المتغير داخل comprehension له نطاقه الخاص في Python 3.

اختبر الفهم

لو استبدلت التعبير number ** 2 بالتعبير number + 10، يتغير التحويل فقط. المصدر وعدد العناصر يظلان كما هما.

02 / اختيار وتحويل

شرط في الآخر ولا في الأول؟

PYTHON

```python
numbers = [1, 2, 3, 4, 5]
even = [n for n in numbers if n % 2 == 0]
print(even)
```

OUTPUT

```
[2, 4]
```

الشرط في النهاية يقرر هل نضم العنصر أم نتخطاه. لذلك طول الناتج قد يكون أقل من المصدر. هنا القيم الفردية لم تدخل الناتج أصلا.

PYTHON

```python
numbers = [1, 2, 3]
labels = ["Even" if n % 2 == 0 else "Odd"
          for n in numbers]
print(labels)
```

OUTPUT

```
['Odd', 'Even', 'Odd']
```

التعبير الشرطي قبل for يحدد قيمة بديلة لكل عنصر. كل عنصر له نتيجة، لذلك الطول هنا كما هو. وجود else جزء من التعبير، وليس فلترا في نهاية comprehension.

التسمية بتساعد

سم القائمة حسب الناتج: even للأعداد المختارة، وlabels للأوصاف. اسم عام مثل result أحيانا يخفي الغرض.

03 / إمتى نفك الاختصار؟

قابلية القراءة أولا

الأوامر ذات الآثار الجانبية

ما تستخدمش comprehension لمجرد print لكل عنصر. هتعمل قائمة من قيم None بدون فائدة. لو المطلوب فعل مثل الطباعة أو الكتابة في ملف، الحلقة العادية غالبا أوضح.

منطق كتير في سطر

لو التحويل محتاج عدة خطوات أو معالجة أخطاء، اكتب دالة واضحة أو حلقة عادية. الاختصار مش مكسب لو لازم تفك السطر كل مرة علشان تفهمه.

القائمة تتكون كاملة

comprehension دي تبني list بكل النتائج في الذاكرة. مناسبة للأمثلة الصغيرة هنا، لكن البيانات الضخمة قد تحتاج طريقة تدرجية مثل generator expression؛ ده موضوع توسع لاحق.

PYTHON

```python
names = [" Ali ", " Mona "]
clean = [name.strip() for name in names]
print(clean)
```

OUTPUT

```
['Ali', 'Mona']
```

strip ترجع نصا جديدا بعد إزالة المسافات من الطرفين. هنا التحويل قصير ومباشر، لذلك comprehension مناسبة.

راجع الحالات

مصدر فارغ ينتج قائمة فارغة. ولو كل العناصر فشلت في الفلتر، النتيجة فاضية برضه؛ ده مش خطأ.

جرب قبل ما تبص على الحل

دورك إنت

تطبيق 01 / ضاعف القيم

كون قائمة ضعف كل عدد في [2, 4, 6].

تلميح صغير

ضع عملية الضرب قبل for.

تطبيق 02 / اختار الموجب

اختر القيم الأكبر من صفر من [-1, 0, 3, 5].

تلميح صغير

الفلتر في النهاية.

تطبيق 03 / بديل لكل عنصر

استبدل الأعداد السالبة بصفر في [-2, 4, -1] مع الاحتفاظ بالطول.

تلميح صغير

تعبير شرطي قبل for، وليس فلترا.

قبل صفحة الحل: جرب حالة عادية وحالة على الحدود. اكتب النتيجة المتوقعة الأول، وبعدها قارنها باللي طلع فعلا.

مساحة لتجربتك

حلول مشروحة / قارن طريقة التفكير

نراجع اللي عملناه

حل 01 / ضاعف القيم

```python
print([n * 2 for n in [2, 4, 6]])
```

الناتج [4, 8, 12].

حل 02 / اختار الموجب

```python
print([n for n in [-1, 0, 3, 5] if n > 0])
```

الصفر لا يدخل لأن الشرط أكبر من، وليس أكبر أو يساوي.

حل 03 / بديل لكل عنصر

```python
values = [-2, 4, -1]
print([0 if n < 0 else n for n in values])
```

الناتج ثلاث قيم: [0, 4, 0].

ورقة ترجع لها بسرعة

قبل ما تقفل الدفتر

راجع نفسك

- أقرأ التعبير والمصدر والشرط منفصلين.
- أفرق بين الفلترة والاستبدال.
- أرجع للحلقة لما الاختصار يعقد القراءة.

المصطلحات من غير تعقيد

Comprehension : صيغة لبناء مجموعة من عناصر مصدر.

Filter : اختيار العناصر المطابقة لشرط.

Transform : حساب قيمة جديدة من العنصر.

اختبار الفهم

هل قائمة النتائج دائما بنفس طول المصدر؟ فقط لو لا يوجد فلتر يزيل عناصر.

نكمل على إيه؟

الدفتر 13: tuples وتسلسلات لا نغير مواقع عناصرها.

لما تحتاج التفاصيل الرسمية

الشرح والأمثلة هنا مكتوبة للمبتدئ. الروابط التالية مراجع للمراجعة والتوسع، وليست نصا منقولا منها.

هياكل البيانات في Python

https://docs.python.org/3/tutorial/datastructures.html